Chapter 4: Joining Data With Pandas

Lesson 1: Data Merging Basics

Inner Joins

- **Understanding DataFrames and Tables**: You discovered that in pandas, tables are represented as DataFrames, and merging them is a crucial skill for data analysis.
- **The** `merge()` **Method**: You learned how to use pandas' `merge()` method to join two DataFrames. For example, to merge two DataFrames, `df1` and `df2`, on a common column named 'common_column', you would use `df1.merge(df2, on='common_column')`.
- **Inner Join Mechanics**: An inner join returns a DataFrame containing only the rows that have matching values in both tables. This was illustrated through merging ward and census data, where only wards present in both DataFrames were included in the result.
- **Suffixes in Merged DataFrames**: You saw how pandas handles overlapping column names by appending suffixes, and you learned to customize these suffixes using the `suffixes` argument in the `merge()` method to make the output clearer.
- **Practical Application**: Through exercises, you applied these concepts by merging Chicago taxi owners and vehicle tables on the 'vid' column to analyze taxi fuel types, demonstrating the real-world utility of inner joins.

One-to-many Relationships

- The difference between one-to-one and one-to-many relationships. In a one-to-one relationship, each row in the first table corresponds to exactly one row in the second table. Conversely, in a one-to-many relationship, a single row in the first table can relate to multiple rows in the second table.
- How to merge DataFrames using the `merge` method in pandas, particularly when dealing with one-to-many relationships. You saw how the `merge` method requires specifying the column to join on, which in your example was the 'ward' column linking ward data to business licenses.
- The practical implications of merging tables with one-to-many relationships, such as the significant increase in the number of rows in the resulting DataFrame. This was demonstrated by merging ward information with business licenses, resulting in a DataFrame much larger than the original wards table.
- Through exercises, you gained hands-on experience by merging tables with one-to-many relationships and analyzing the merged data. For instance, you merged a `licenses` table with a `biz_owners` table on the 'account' column and then analyzed the most common business owner titles.
- Here's a snippet from your exercises that illustrates how to merge tables and analyze the results:

```
# Merge the licenses and biz_owners table on account
```

```
licenses_owners = licenses.merge(biz_owners, on='account')

# Group the results by title then count the number of accounts
counted_df = licenses_owners.groupby('title').agg({'account':'count'})

# Sort the counted_df in descending order
sorted_df = counted_df.sort_values(by='account', ascending=False)

# Use .head() method to print the first few rows of sorted_df
print(sorted_df.head())
```

Merging Multiple DataFrames

- The concept of merging tables with a one-to-many relationship using the `merge` method, which is essential for bringing together disparate data sources.
- The importance of selecting the correct keys (columns) for merging tables to avoid duplicating or losing data. For example, merging on both address and zip code to ensure accurate matching of records.
- How to merge more than two tables by chaining the `merge` method, and the significance of adding suffixes to differentiate columns with the same name from different tables.
- The practical application of these techniques through exercises, including merging business licenses, grants, and ward information to analyze grant distribution across wards.

In one of the exercises, you practiced merging three tables (`licenses`, `zip_demo`, and `wards`) and then grouping the results to find the median income by alderman:

```
licenses_zip_ward = licenses.merge(zip_demo, on='zip') \
                            .merge(wards, on='ward')
print(licenses_zip_ward.groupby('alderman').agg({'income':'median'}))
```

Lesson 2: Merging Tables With Different Join Types

Left Joins

- A left join is performed using the `merge` method in pandas, specifying `'left'` in the `how` argument.
- You saw how a left join keeps all rows from the left table, filling in missing values from the right table with NaN (Not a Number) to indicate missing data.

- Through examples, you learned that after merging two tables with a left join, the resulting table includes all rows from the left table. If the relationship between tables is one-to-one, the number of rows in the resulting table matches the left table.
- You practiced identifying missing data in a dataset by merging the `movies` table with the `financials` table, showcasing how left joins can be used to find which movies are missing financial data.
- You also enriched a dataset by adding marketing taglines to a sample of movie data from the Toy Story series, demonstrating the practical application of left joins in enhancing datasets with additional information.

Here's a code snippet that illustrates how to perform a left join:

```
merged_data = movies.merge(taglines, on='ID', how='left')
```

Other Joins

- **Right Join**: You discovered that a right join returns all rows from the right table and those rows from the left table where the key columns match. It's useful for ensuring no data is missing from the right table in the result.
- **Outer Join**: You learned that an outer join returns all rows from both tables, filling in with nulls where there is no match. This join type is particularly useful for finding unmatched data between two tables.
- **Merging with Different Column Names**: You explored how to merge tables when the key columns have different names using the `left_on` and `right_on` parameters in the `merge` method.

Here's an example code snippet you worked with:

```
# Use right join to merge the movie_to_genres and pop_movies tables
genres_movies = movie_to_genres.merge(pop_movies, how='right',
                                      left_on='movie_id',
                                      right_on='id')
```

Merging indexes

- **Self Join Basics**: A self join is used when you need to merge a table with itself to compare values within the same table. For example, linking movies to their sequels by matching a movie's sequel ID with another movie's ID.
- **Inner Join on Self**: Initially, you performed an inner join on the table, which resulted in a table where each movie and its sequel were listed in one row. Movies without sequels, like Avatar and Titanic, were excluded.

- **Left Join Modification**: By changing the merge to a left join, you included all original movies in the result, showing sequel information where available. This demonstrated how movies without direct sequels could still be included in the output.
- **Practical Application**: You explored how self joins could be applied beyond movies, such as hierarchical relationships in employee-manager scenarios, sequential relationships, or network graphs.
- **Code Implementation**: You used the `.merge()` method with `left_on` and `right_on` attributes to specify the columns for matching, and `suffixes` to differentiate between original and sequel movies in the result.

```
sequels.merge(sequels, left_on='sequel', right_on='id', suffixes=('_org', '_seq'))
```

Lesson 3: Advance Merging and Concatenating

Filtering Joins

- Understanding that DataFrame indexes, often unique IDs, can serve as a basis for merging tables, enhancing the flexibility and efficiency of data manipulation.
- Learning to merge tables using their indexes instead of columns. For example, using `movies.merge(taglines, on='id', how='left')` merges the `movies` and `taglines` tables based on their `id` index.
- Discovering how to handle merges with multiIndex data structures, where you merge on multiple levels of indexes, such as both movie ID and cast ID.
- Addressing scenarios where index level names differ between tables by using `left_on` and `right_on` arguments in the merge method, alongside `left_index` and `right_index` set to True, to specify index merging.

Practical exercises reinforced these concepts:

- You merged the `movies` and `ratings` tables on their index, ensuring all rows from `movies` were included, using `movies.merge(ratings, on='id', how='left')`.
- Investigating whether sequels earn more than their originals involved merging modified `sequels` and `financial` tables on their index, then comparing revenue figures to find differences.

Lesson 4: Merging Ordered and Time-Series Data

Filtering Joins

- **Semi-joins**: You discovered that a semi-join filters the left DataFrame to only those rows that have a matching value in the right DataFrame. Unlike an inner join, it returns just the columns from the left DataFrame without duplicates. For instance, to find genres present

in a table of top songs, you first performed an inner join and then used the `isin()` method to filter the genres.
- **Anti-joins**: You learned that an anti-join returns rows from the left DataFrame that do not have a matching row in the right DataFrame, again only returning columns from the left DataFrame. This was demonstrated by finding genres not present in the top tracks table, using a left join with the `indicator` argument set to True, and then filtering with the `_merge` column.
- **Concatenating DataFrames**: The lesson covered using `pandas.concat` to vertically combine DataFrames, which is crucial when you want to append rows from one DataFrame to another.
- **Data validation**: You learned techniques for validating your combined data structures to ensure accuracy and integrity in your data analysis.

Here's a snippet of code you worked with, demonstrating how to perform a semi-join:

```
# Merge the non_mus_tck and top_invoices tables on tid
tracks_invoices = non_mus_tcks.merge(top_invoices, on='tid')

# Use .isin() to subset non_mus_tcks to rows with tid in tracks_invoices
top_tracks = non_mus_tcks[non_mus_tcks['tid'].isin(tracks_invoices['tid'])]

# Group the top_tracks by gid and count the tid rows
cnt_by_gid = top_tracks.groupby(['gid'], as_index=False).agg({'tid':'count'})

# Merge the genres table to cnt_by_gid on gid and print
print(cnt_by_gid.merge(genres, on='gid'))
```

CONCATENATING DATAFRAMES VERTICALLY

pd.concat() can concatenate both vertically and horizontally. Use axis=0 for vertical (default).

Basic concatenation:

```
pd.concat([inv_jan, inv_feb, inv_mar])
```

Note: Original indexes are preserved, so you may get repeated index values (0, 1, 2, 0, 1, 2...).

Ignore the index (reset to continuous):

```
pd.concat([inv_jan, inv_feb, inv_mar], ignore_index=True)
```

Add labels to track source tables:

```
pd.concat([inv_jan, inv_feb, inv_mar],
 ignore_index=False,
 keys=['jan', 'feb', 'mar'])
```

Concatenating tables with different column names:

With sort=True — includes all columns, fills missing values with NaN:

```
pd.concat([inv_jan, inv_feb],
sort=True)
```

With join='inner' — only keeps shared columns, no NaN:

```
pd.concat([inv_jan, inv_feb],
join='inner')
```

VERIFYING INTEGRITY

Common issues to watch for:

- Merging: Unintentional one-to-many or many-to-many relationships
- Concatenating: Duplicate records unintentionally introduced

Validating merges:

```
.merge(validate=None)
```

Options: 'one_to_one', 'one_to_many', 'many_to_one', 'many_to_many'

Example — will raise MergeError if not truly one-to-one:

```
tracks.merge(specs, on='tid', validate='one_to_one')
```

Example — valid one-to-many merge:

```
albums.merge(tracks, on='aid', validate='one_to_many')
```

Verifying concatenations:

```
pd.concat([inv_feb, inv_mar], verify_integrity=True)
```

Raises a ValueError if the new index has duplicate values. The default is False (no check).

Why verify and what to do:

- Real world data is often NOT clean
- Fix incorrect data or drop duplicate rows

- **Merging Tables**: You discovered how to use the `validate` argument in the `.merge()` method to check the relationship between two tables. For instance, specifying `validate="one_to_one"` ensures that the merge operation adheres to a one-to-one relationship, raising a `MergeError` if this condition is not met due to duplicates in the merging column.

```
merged_data = tracks.merge(specs, on="tid", validate="one_to_one")
```

- **Concatenating Tables**: You explored the `verify_integrity` argument of the `pandas.concat()` function, which, when set to `True`, checks for duplicate index values across the concatenated tables. This helps prevent unintentional duplication of records, ensuring the uniqueness of index values in the resulting DataFrame.
- **Handling Errors**: Through examples, you learned that encountering a `MergeError` or `ValueError` indicates issues like duplicate keys or overlapping index values. These errors serve as a prompt to clean or adjust the data, such as removing duplicates, before proceeding with merge or concatenate operations.
- **Practical Application**: You applied these concepts in exercises, merging tables with different relationship types (one-to-one, one-to-many) and concatenating tables while checking for index integrity. This hands-on practice reinforced the importance of data validation in real-world data manipulation tasks.

Lesson 4: **Merging Ordered and Time-Series Data**

Using merge_ordered()

- `merge_ordered()` is similar to the standard merge method but with an outer join by default and results in sorted output.
- It supports merging on different columns, various types of joins, and adding suffixes for overlapping column names.
- Unlike the `merge` method, which is called on a DataFrame, `merge_ordered()` is called directly from pandas, e.g., `pd.merge_ordered()`.
- You explored merging stock price data for Apple and McDonald's, highlighting how to handle missing data using the forward fill method (`ffill`). This technique fills missing values with the previous value, which is particularly useful in time-series data where missing values can distort analyses.

In practice exercises, you applied merge_ordered() to explore economic theories and relationships, such as the correlation between the S&P 500 returns and US GDP, and the Phillips curve, which examines the inverse relationship between inflation and unemployment. Here's a snippet from one of the exercises:

```
inflation_unemploy = pd.merge_ordered(inflation, unemployment,
                                      on='date', how='inner')
print(inflation_unemploy)
inflation_unemploy.plot(kind='scatter', x='unemployment_rate', y='cpi')
plt.show()
```

Using merge_asof()

- Similar to a merge_ordered()
    - left join
- Similar features as
    - merge_ordered()
- Match on the nearest key column and not exact matches.
- Merged "on" columns must be sorted.

```
pd.merge_asof(visa, ibm, on='date_time',
          suffixes=('_visa', '_ibm'))

pd.merge_asof(visa, ibm, on=['date_time'],
          suffixes=('_visa', '_ibm'),
          direction='forward')
```

When to use merge_asof()

- Data sampled from a process
- Developing a training set (no data leakage)